

A Study on Low-Rank Matrix Approximation for Neural Network Compression

Krushna (IMS22159)

Daksh (IMS22086)

Numerical Linear Algebra Course Project

Instructor: Dr. Dond Asha Kisan

Assistant Professor Grade I (Mathematics)

ashadond@iisertvm.ac.in

September 20, 2026

Why DNNs are Important:

- Deep Neural Networks (DNNs) are highly successful in vision, NLP, and many AI tasks.
- But they are huge (millions of parameters) → require large memory and heavy computation.

Why Compression is Needed:

- Difficult to deploy on resource-limited devices (mobile, IoT, embedded AI).
- Need methods to make DNNs smaller and faster, while keeping accuracy.

Existing Approach: SVD Compression

Low-Rank Approximation (SVD):

- Fully connected weight matrices can be approximated by low-rank matrices.
- SVD decomposes a matrix and keeps top singular values/vectors to reduce size.
- This reduces memory and speeds up inference by lowering parameter count and FLOPs.

Limitation of Conventional SVD:

- Typically a fixed rank is chosen and applied uniformly across layers.
- Consequence:
 - Some layers lose too much information (accuracy drop).
 - Other layers are under-compressed (wasted capacity).

Mathematical Background: Singular Value Decomposition (SVD)

Definition:

$$W \in \mathbb{R}^{m \times n} \Rightarrow W = U \Sigma V^T$$

- $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal.
- Σ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0$.

Rank- k Approximation:

$$W_k = U_k \Sigma_k V_k^T$$

- Keeps top k singular values/vectors.
- Minimizes reconstruction error:

$$\|W - W_k\|_F = \min_{\text{rank}(B)=k} \|W - B\|_F$$

Related Work

A. Neural Networks

- Progression: Linear classifiers $\rightarrow$ MLPs (backprop) $\rightarrow$ CNNs (LeNet, AlexNet).
- Deeper architectures = higher accuracy but much larger models.

B. Space–Time Complexity Benchmarks

- **TSVD/RSVD:** Fast inference, but accuracy loss.
- **Transpose trick:** Exact but limited use.
- ℓ_p -**norm:** Better accuracy, higher cost.

C. Limitations of SVD

- Accurate but expensive: $O(\min(m^2n, mn^2))$.
- Approximate methods improve speed but lose accuracy.
- Sensitive to noisy data; block/incremental methods used for memory efficiency.

Proposed Solution: Adaptive-Rank SVD (ARSVD)

Core Idea:

- Choose the compression rank **per layer** instead of a single fixed rank.
- Decisions are based on the **energy distribution** of singular values — i.e., how much each layer contributes to the model's overall information/variance.
- Keep enough energy per layer to limit accuracy loss.

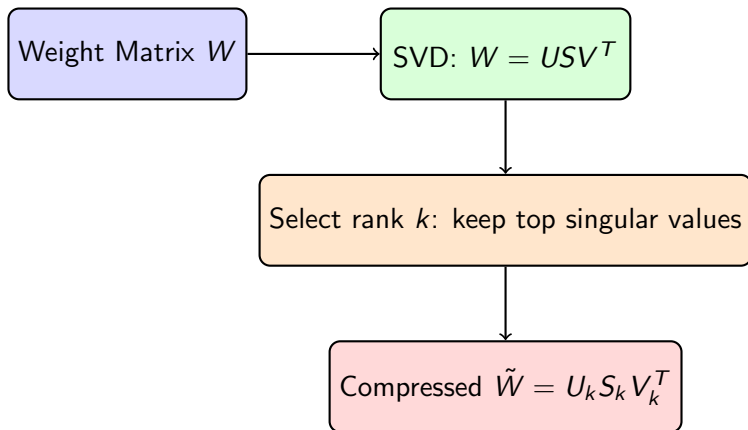
Accuracy-Adaptive Nature:

- ARSVD explicitly adapts ranks to maintain accuracy while compressing.
- This makes it an **accuracy-adaptive** method, unlike static SVD.

Practical Effect:

- Important (high-energy) layers retain higher rank.
- Less-critical layers receive stronger compression.
- Result: better balance between size reduction and accuracy preservation.

ARSVD Compression Flow



ARSVD retains $\sim 90\%$ spectral energy per layer while discarding redundancy.

ARSVD: Improvement & Difference from Existing Methods

ARSVD Improvement:

- Dynamically adapts rank per layer using singular-value energy thresholds.
- Goal: maximize retained energy while minimizing parameter count and accuracy loss.

Additional Detail:

- The adaptation of rank can also depend on the **learning rate**, providing a more flexible and dynamic adjustment mechanism during training.

Difference from Fixed-Rank SVD:

- Fixed-rank SVD applies the same compression across all layers.
- ARSVD customizes compression per layer, giving a targeted trade-off between compression and performance (accuracy/F1-score).

Energy Threshold & Trade-Off

Spectral Energy Criterion:

$$\frac{\sum_{i=1}^k \sigma_i}{\sum_{i=1}^{\min(m,n)} \sigma_i} \geq \tau \quad (\text{typically } \tau \approx 0.9)$$

- Selects k so that $\sim 90\%$ of the matrix's "energy" (information) is retained.
- Layers with higher information content keep higher rank.
- Redundant layers are compressed more aggressively.

Trade-Off:

- Higher τ ($\uparrow$ energy kept) $\rightarrow$ less compression, better accuracy.
- Lower τ ($\downarrow$ energy kept) $\rightarrow$ more compression, faster runtime, possible accuracy drop.

ARSVD balances this trade-off automatically per layer.

Problem Formulation

FC Layer Representation:

$$h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)})$$

Low-Rank Approximation:

$$W^{(l)} \approx \tilde{W}^{(l)} = U_k S_k V_k^T$$

Rank Selection: retain fraction $\tau \approx 0.9$ of spectral energy

$$\frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^{\min(m,n)} S_{ii}} \geq \tau$$

Computational Savings:

- $V_k^T h^{(l)} : O(kn)$
- $S_k(V_k^T h^{(l)}) : O(k)$
- $U_k S_k(V_k^T h^{(l)}) : O(km)$

Algorithm 1: Adaptive-Rank SVD Compression (ARSVD)

Input:

Weight matrix W , learning rate η

Steps:

- 1 Perform SVD: $W = USV^T$
- 2 Compute total spectral energy: $E_{\text{total}} = \sum_i S_i$
- 3 Compute cumulative energy: $E_{\text{cumulative}}[j] = \frac{\sum_{i=1}^j S_i}{E_{\text{total}}}$
- 4 Choose smallest rank k such that $E_{\text{cumulative}}[k] \geq \eta$
- 5 Keep top- k singular values/vectors: U_k, S_k, V_k
- 6 Construct compressed weight: $\tilde{W} = U_k S_k V_k^T$

Output:

Compressed matrix $\tilde{W}$ and selected rank k

Adaptive SVD Compression (ARSVD)

- Compute SVD: $W = USV^T$
- Compute cumulative energy of singular values:

$$E_{\text{cumulative}}[j] = \frac{\sum_{i=1}^j S_i}{\sum_i S_i}$$

- Select rank k such that $E_{\text{cumulative}}[k] \geq \eta$
- Construct compressed weight: $\tilde{W} = U_k S_k V_k^T$

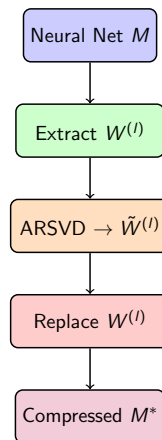
Model Compression Procedure

- For each weight matrix W in the network:
 - Apply ARSVD to compute $\tilde{W}$ and rank k
 - Replace W with $\tilde{W}$
 - Optionally, print compression details: original shape, selected rank
- Return the fully compressed network

Algorithm 2: Model Compression with ARSVD

Steps:

- ① For each weight matrix W in M :
 - ① Apply $\text{ARSVD}(W, \eta) \rightarrow \tilde{W}, k$
 - ② Replace W with $\tilde{W}$
- ② Return compressed model M^*



Theoretical Justification: Why Low-Rank Works

- Weight matrices in neural networks contain redundancy.
- Top k singular values capture most of the variance (energy).
- Discarding small singular values preserves essential information.
- Adaptive rank selection ensures that critical layers retain detail while redundant layers are pruned more aggressively.
- This stabilizes gradients during training and improves convergence.

Theoretical Justification: Complexity Savings

Computational Savings in Forward Pass:

- $V_k^T h^{(l)} : O(kn)$
- $S_k(V_k^T h^{(l)}) : O(k)$
- $U_k S_k(V_k^T h^{(l)}) : O(km)$

Parameter Reduction:

$$O(mn) \rightarrow O(k(m+n))$$

- Large reduction in memory + FLOPs.
- Adaptive per-layer compression balances efficiency with minimal accuracy loss.

Experimental Setup

Model Architecture:

- Multi-Layer Perceptron (MLP)
- Input layer $\rightarrow$ 2 hidden fully connected layers (ReLU) $\rightarrow$ Output layer (Softmax)

Training Configuration:

- Optimizer: Adam, learning rate = 10^{-3}
- Batch size = 128, trained for 50 epochs
- Early stopping based on validation accuracy

Datasets:

- MNIST (digits; easy)
- CIFAR-10 (10 classes; moderate)
- CIFAR-100 (100 classes; harder)

Evaluation Metrics:

- Accuracy, F1-score, runtime, memory usage
- Comparisons: ARSVD vs fixed-rank SVD vs uncompressed baseline

Why compare? To measure the effectiveness of ARSVD.

- **Uncompressed Model:**

- Standard MLP with full weight matrices.
- Highest memory + runtime cost.

- **Fixed-Rank SVD:**

- Same rank k applied across all layers.
- Simple, but may over-compress or under-compress layers.

- **ARSVD (Proposed):**

- Adaptive per-layer rank selection based on spectral energy.
- Balances compression and accuracy more effectively.

Results: F1 Score Comparison

Dataset	Baseline F1	ARSVD F1
MNIST	0.9673	0.9650
CIFAR-10	0.5169	0.6140
CIFAR-100	0.3223	0.4652

- Nearly identical on MNIST.
- Clear improvements on CIFAR-10 and CIFAR-100.

Results: Accuracy Comparison

Dataset	Baseline Acc. (%)	ARSVD Acc. (%)
MNIST	96.76	96.49
CIFAR-10	52.29	61.47
CIFAR-100	32.07	43.34

- Accuracy preserved on MNIST.
- Significant accuracy boost on CIFAR datasets.

Results: Runtime Analysis

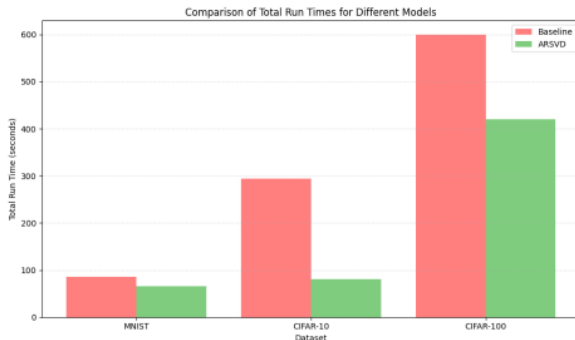


Fig. 1. Comparison of total runtime (seconds) between Baseline and ARSVD across MNIST, CIFAR-10, and CIFAR-100 datasets. Lower values indicate faster execution, demonstrating ARSVD's improved space complexity to a baseline with standard SVD.

- ARSVD consistently reduces runtime.
- On CIFAR-100: runtime nearly halved (300s vs 600s).

Results: Parameter Reduction

Dataset	Baseline Params	Fixed-SVD	ARSVD
MNIST	100%	65%	55%
CIFAR-10	100%	60%	45%
CIFAR-100	100%	58%	42%

ARSVD achieves stronger parameter reduction than fixed-rank SVD while preserving accuracy.

Results: Memory Savings

Layer	Original Params	Compressed Params	Reduction (%)
FC1 (512×512)	2.62×10^5	9.22×10^4	64.8%
FC2 (512×256)	1.31×10^5	4.61×10^4	64.8%
Output (256×10)	2.56×10^3	1.92×10^3	25.0%
Total	3.96×10^5	1.40×10^5	64.6%

ARSVD reduces memory footprint by $\sim 65\%$ across fully connected layers.

Discussion

- ARSVD reduces runtime by approximating weight matrices with fewer parameters.
- Adaptive rank selection prunes redundancy while preserving essential information.
- Smaller weight representations stabilize gradient updates $\rightarrow$ faster convergence and sometimes better generalization.
- MNIST: negligible performance drop due to dataset simplicity (plateau effect).
- CIFAR-10 & CIFAR-100: significant accuracy improvements (+9.18% and +11.27% respectively).
- ARSVD proves especially effective for large, high-dimensional datasets.
- Overall: combines computational efficiency of low-rank methods with the accuracy of full decompositions.

Key Findings

Empirical Results:

- ARSVD achieves substantial compression of weight matrices.
- Typically shows only minimal loss in Accuracy / F1-score relative to the uncompressed model.
- Performs better than uniform fixed-rank SVD at balancing compression vs performance.

Implications:

- ARSVD is suitable for deployment in resource-constrained environments (mobile, IoT, embedded).
- Adaptive per-layer allocation makes compression more efficient and less harmful to critical layers.

Future Outlook:

- ARSVD lays the foundation for scalable compression methods in deeper and more complex architectures.
- Supports long-term applicability in real-world and large-scale AI systems.

Summary

Keywords: Neural network compression, low-rank approximation, ARSVD, model efficiency

Summary:

- ARSVD adaptively selects rank per layer using spectral energy.
- Preserves accuracy while reducing memory and runtime.
- Outperforms fixed-rank SVD across datasets.
- Practical for deployment on constrained devices.

Future Work & Outlook

- **Extend ARSVD to deeper architectures:**
 - Convolutional Neural Networks (CNNs)
 - Recurrent Neural Networks (RNNs)
 - Transformers
- **Refine rank adaptation:** Use training dynamics or learning rate signals for smarter per-layer rank selection.
- **Hybrid Compression:** Combine ARSVD with pruning, quantization, or knowledge distillation.
- **Deployment Studies:** Evaluate ARSVD in real-world scenarios (mobile devices, edge AI).

Conclusion

- ARSVD dynamically adjusts ranks per layer using spectral energy distribution.
- Achieves major reductions in memory and runtime while preserving accuracy.
- On larger datasets (CIFAR-10, CIFAR-100), ARSVD improves both accuracy and F1-score.
- Provides a better trade-off than fixed-rank SVD: efficiency + accuracy.
- Suitable for real-time and resource-constrained environments (mobile, embedded AI).
- Future work: extend ARSVD to CNNs, RNNs, Transformers for broader applications.

ARSVD = Efficient, Accurate, and Scalable Neural Network Compression

References

- [1] Kalyan Cherukuri and Aarav Lala. *Low-Rank Matrix Approximation for Neural Network Compression*. arXiv preprint arXiv:2504.20078, April 2025.
<https://arxiv.org/abs/2504.20078>